

Flask Revision Notes

1. Introduction

Flask is a lightweight Python web framework used for web apps and REST APIs. It follows WSGI and uses Jinja2 templates.

2. Project Structure

Typical folders: app.py, templates/, static/, models/, routes/, services/, utils/, config.py.

3. Routing

Routes map URLs to view functions using decorators. Dynamic routes accept parameters.

4. Request & Response

Request contains headers, params, forms and JSON. Responses can be HTML, JSON, redirects or files.

5. Templates

Jinja2 renders dynamic HTML with variables, loops, conditions and template inheritance.

6. Database

SQLAlchemy ORM maps Python classes to database tables. Common CRUD operations: Create Read Update Delete.

7. REST APIs

Use GET POST PUT PATCH DELETE. Return JSON responses and status codes.

8. Authentication

Sessions, cookies and JWT tokens are common authentication methods.

9. Deployment & Performance

Deploy with Gunicorn + Nginx on Linux or platforms like Render/Railway. Use caching and indexing.

10. Interview Revision

Remember app instance, routes, request lifecycle, blueprints, extensions, ORM, authentication, error handlers, testing.

1. Introduction

Flask is a lightweight Python web framework used for web apps and REST APIs. It follows WSGI and uses Jinja2 templates.

2. Project Structure

Typical folders: app.py, templates/, static/, models/, routes/, services/, utils/, config.py.

3. Routing

Routes map URLs to view functions using decorators. Dynamic routes accept parameters.

4. Request & Response

Request contains headers, params, forms and JSON. Responses can be HTML, JSON, redirects or files.

5. Templates

Jinja2 renders dynamic HTML with variables, loops, conditions and template inheritance.

6. Database

SQLAlchemy ORM maps Python classes to database tables. Common CRUD operations: Create Read Update Delete.

7. REST APIs

Use GET POST PUT PATCH DELETE. Return JSON responses and status codes.

8. Authentication

Sessions, cookies and JWT tokens are common authentication methods.

9. Deployment & Performance

Deploy with Gunicorn + Nginx on Linux or platforms like Render/Railway. Use caching and indexing.

10. Interview Revision

Remember app instance, routes, request lifecycle, blueprints, extensions, ORM, authentication, error handlers, testing.

1. Introduction

Flask is a lightweight Python web framework used for web apps and REST APIs. It follows WSGI and uses Jinja2 templates.

2. Project Structure

Typical folders: app.py, templates/, static/, models/, routes/, services/, utils/, config.py.

3. Routing

Routes map URLs to view functions using decorators. Dynamic routes accept parameters.

4. Request & Response

Request contains headers, params, forms and JSON. Responses can be HTML, JSON, redirects or files.

5. Templates

Jinja2 renders dynamic HTML with variables, loops, conditions and template inheritance.

6. Database

SQLAlchemy ORM maps Python classes to database tables. Common CRUD operations: Create Read Update Delete.

7. REST APIs

Use GET POST PUT PATCH DELETE. Return JSON responses and status codes.

8. Authentication

Sessions, cookies and JWT tokens are common authentication methods.

9. Deployment & Performance

Deploy with Gunicorn + Nginx on Linux or platforms like Render/Railway. Use caching and indexing.

10. Interview Revision

Remember app instance, routes, request lifecycle, blueprints, extensions, ORM, authentication, error handlers, testing.

1. Introduction

Flask is a lightweight Python web framework used for web apps and REST APIs. It follows WSGI and uses Jinja2 templates.

2. Project Structure

Typical folders: app.py, templates/, static/, models/, routes/, services/, utils/, config.py.

3. Routing

Routes map URLs to view functions using decorators. Dynamic routes accept parameters.

4. Request & Response

Request contains headers, params, forms and JSON. Responses can be HTML, JSON, redirects or files.

5. Templates

Jinja2 renders dynamic HTML with variables, loops, conditions and template inheritance.

6. Database

SQLAlchemy ORM maps Python classes to database tables. Common CRUD operations: Create Read Update Delete.

7. REST APIs

Use GET POST PUT PATCH DELETE. Return JSON responses and status codes.

8. Authentication

Sessions, cookies and JWT tokens are common authentication methods.

9. Deployment & Performance

Deploy with Gunicorn + Nginx on Linux or platforms like Render/Railway. Use caching and indexing.

10. Interview Revision

Remember app instance, routes, request lifecycle, blueprints, extensions, ORM, authentication, error handlers, testing.

1. Introduction

Flask is a lightweight Python web framework used for web apps and REST APIs. It follows WSGI and uses Jinja2 templates.

2. Project Structure

Typical folders: app.py, templates/, static/, models/, routes/, services/, utils/, config.py.

3. Routing

Routes map URLs to view functions using decorators. Dynamic routes accept parameters.

4. Request & Response

Request contains headers, params, forms and JSON. Responses can be HTML, JSON, redirects or files.

5. Templates

Jinja2 renders dynamic HTML with variables, loops, conditions and template inheritance.

6. Database

SQLAlchemy ORM maps Python classes to database tables. Common CRUD operations: Create Read Update Delete.

7. REST APIs

Use GET POST PUT PATCH DELETE. Return JSON responses and status codes.

8. Authentication

Sessions, cookies and JWT tokens are common authentication methods.

9. Deployment & Performance

Deploy with Gunicorn + Nginx on Linux or platforms like Render/Railway. Use caching and indexing.

10. Interview Revision

Remember app instance, routes, request lifecycle, blueprints, extensions, ORM, authentication, error handlers, testing.

1. Introduction

Flask is a lightweight Python web framework used for web apps and REST APIs. It follows WSGI and uses Jinja2 templates.

2. Project Structure

Typical folders: app.py, templates/, static/, models/, routes/, services/, utils/, config.py.

3. Routing

Routes map URLs to view functions using decorators. Dynamic routes accept parameters.

4. Request & Response

Request contains headers, params, forms and JSON. Responses can be HTML, JSON, redirects or files.

5. Templates

Jinja2 renders dynamic HTML with variables, loops, conditions and template inheritance.

6. Database

SQLAlchemy ORM maps Python classes to database tables. Common CRUD operations: Create Read Update Delete.

7. REST APIs

Use GET POST PUT PATCH DELETE. Return JSON responses and status codes.

8. Authentication

Sessions, cookies and JWT tokens are common authentication methods.

9. Deployment & Performance

Deploy with Gunicorn + Nginx on Linux or platforms like Render/Railway. Use caching and indexing.

10. Interview Revision

Remember app instance, routes, request lifecycle, blueprints, extensions, ORM, authentication, error handlers, testing.

1. Introduction

Flask is a lightweight Python web framework used for web apps and REST APIs. It follows WSGI and uses Jinja2 templates.

2. Project Structure

Typical folders: app.py, templates/, static/, models/, routes/, services/, utils/, config.py.

3. Routing

Routes map URLs to view functions using decorators. Dynamic routes accept parameters.

4. Request & Response

Request contains headers, params, forms and JSON. Responses can be HTML, JSON, redirects or files.

5. Templates

Jinja2 renders dynamic HTML with variables, loops, conditions and template inheritance.

6. Database

SQLAlchemy ORM maps Python classes to database tables. Common CRUD operations: Create Read Update Delete.

7. REST APIs

Use GET POST PUT PATCH DELETE. Return JSON responses and status codes.

8. Authentication

Sessions, cookies and JWT tokens are common authentication methods.

9. Deployment & Performance

Deploy with Gunicorn + Nginx on Linux or platforms like Render/Railway. Use caching and indexing.

10. Interview Revision

Remember app instance, routes, request lifecycle, blueprints, extensions, ORM, authentication, error handlers, testing.

1. Introduction

Flask is a lightweight Python web framework used for web apps and REST APIs. It follows WSGI and uses Jinja2 templates.

2. Project Structure

Typical folders: app.py, templates/, static/, models/, routes/, services/, utils/, config.py.

3. Routing

Routes map URLs to view functions using decorators. Dynamic routes accept parameters.

4. Request & Response

Request contains headers, params, forms and JSON. Responses can be HTML, JSON, redirects or files.

5. Templates

Jinja2 renders dynamic HTML with variables, loops, conditions and template inheritance.

6. Database

SQLAlchemy ORM maps Python classes to database tables. Common CRUD operations: Create Read Update Delete.

7. REST APIs

Use GET POST PUT PATCH DELETE. Return JSON responses and status codes.

8. Authentication

Sessions, cookies and JWT tokens are common authentication methods.

9. Deployment & Performance

Deploy with Gunicorn + Nginx on Linux or platforms like Render/Railway. Use caching and indexing.

10. Interview Revision

Remember app instance, routes, request lifecycle, blueprints, extensions, ORM, authentication, error handlers, testing.